

Next gen analytics with Anomaly detection

Introduction

Netflix utilises Prod data on Android Tv app and uses that to predict when crash can happen.

<https://netflixtechblog.com/formulating-out-of-memory-kill-prediction-on-the-netflix-app-as-a-machine-learning-problem-989599029109>

What can we do?

- Monitor realtime health metrics of app
- Detect anomalies with respect to page load times, cold start, warm start etc
 - App versions
 - Platform
 - OS version
- Detect device & model specific issues
- Automated evaluation of new build against old on core perf metrics
- Exploratory app profiling (w/o test suite). Orchestrator runs through the app, captures perf metrics, navigates across screen
- Identify issues with page configurations, some examples like
 - Memory spikes because of an incorrect Gif etc
 - Misconfigured pages
 - Invalid images etc ...
- Detect Instrumentation issues
 - Detect if analytics events being fired or not
 - Accidental drops or increases
- Collect signals for how and when can app crash (and other use cases)
 - AOP backed log enrichment
 - Profiling metrics for some users, like peak mem when it crashed
- Enhance logs
 - RN crashes have very limited information
 - Capture user flows
 - Faster issue detection and RCA. Search issue example
- Collect method level metrics
 - Isolate long running operations, traces.
 - Identify degradation
 - Sort the dump collected. Tag > 16ms methods. Target <=11ms for 90Hz

What are we targeting

- Performance metrics tracking
- Instrumentation anomalies

In the later phases, we can pick more items from the list mentioned above

When do we run it?

- CI CD pipeline on develop / release branch merges
- Daily or hourly runs on Production builds

Kinds of Runs

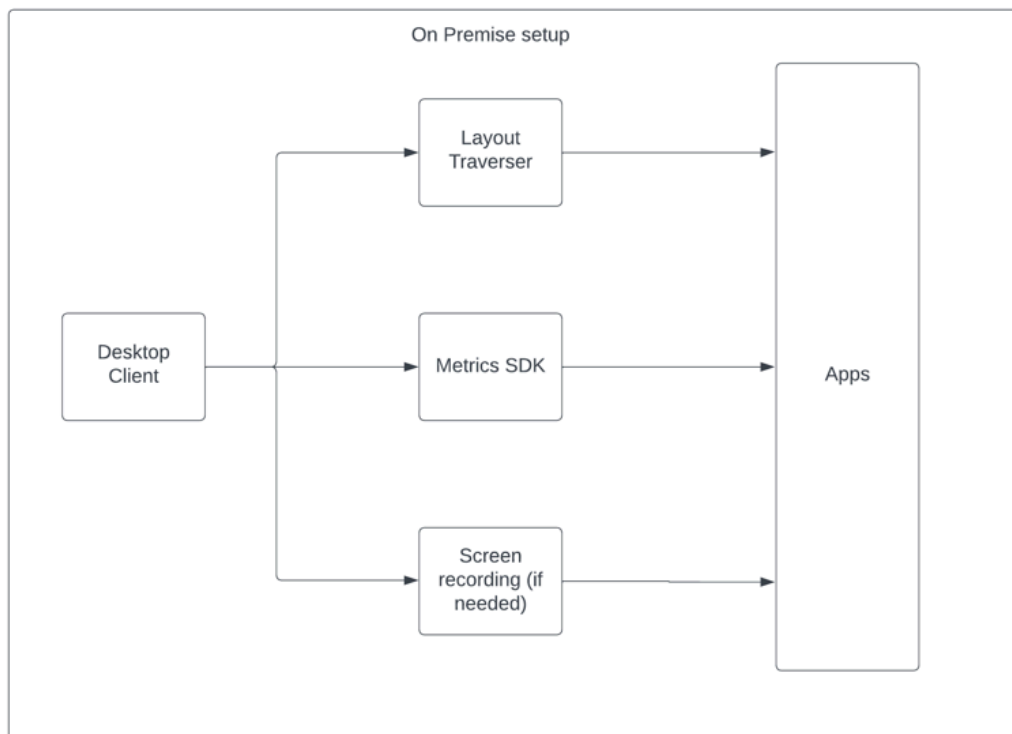
2 kinds of runs to be done

1. On premise
2. Prod user data setup

On Premise Setup

- Here the runs are done on Headspin device farm.
- We do this in our controlled environment.
- Test devices are used. So that the benchmarking and deviations are easier to observe

This will require less infra cost than the Prod setup. Eventually both On Premise and Prod Setup will coexist in the future.



Prod User Data Setup

Nothing can replace the variations and behaviors which users can generate.

- This data will help us in understanding when and why issues occur.
- Can help in detecting issues and patterns for specific OS versions, manufacturers etc
- Some of the data around what memory values to use across a specific kind of device etc can be powered by this. Real world data helps in making sure parameters are tuned appropriately.
- Another place where this can come in handy is when we want to debug an issue for a specific user. Enable user for full debug logs and let the detailed method level traces and other detailed logs come to help us understand what is happening.

Aspects of Anomaly detection

In On Premise Setup, there are a few elements

- Metrics capture
- FE navigation
- Data analysis
- Report

Metrics capture

Samples to be collected from app

Many attributes can be collected from the app including

- FPS metrics
- CPU metrics
- Memory metrics
- MA events
- Navigation tracking
- Method level traces
- And many more details
- Screen load

Attribute through UnHandledExceptionHandler on Android, equivalent in RN & iOS

- Screen
 - Screen load times
 - Fps
 - Other metrics defined earlier tagged against a screen
- Crashes
 - Anomalies on manufacturer

- Android version
- Particular mode
- Background or foreground
- Number of crashes in the period

Crashes vs Anomaly sample data

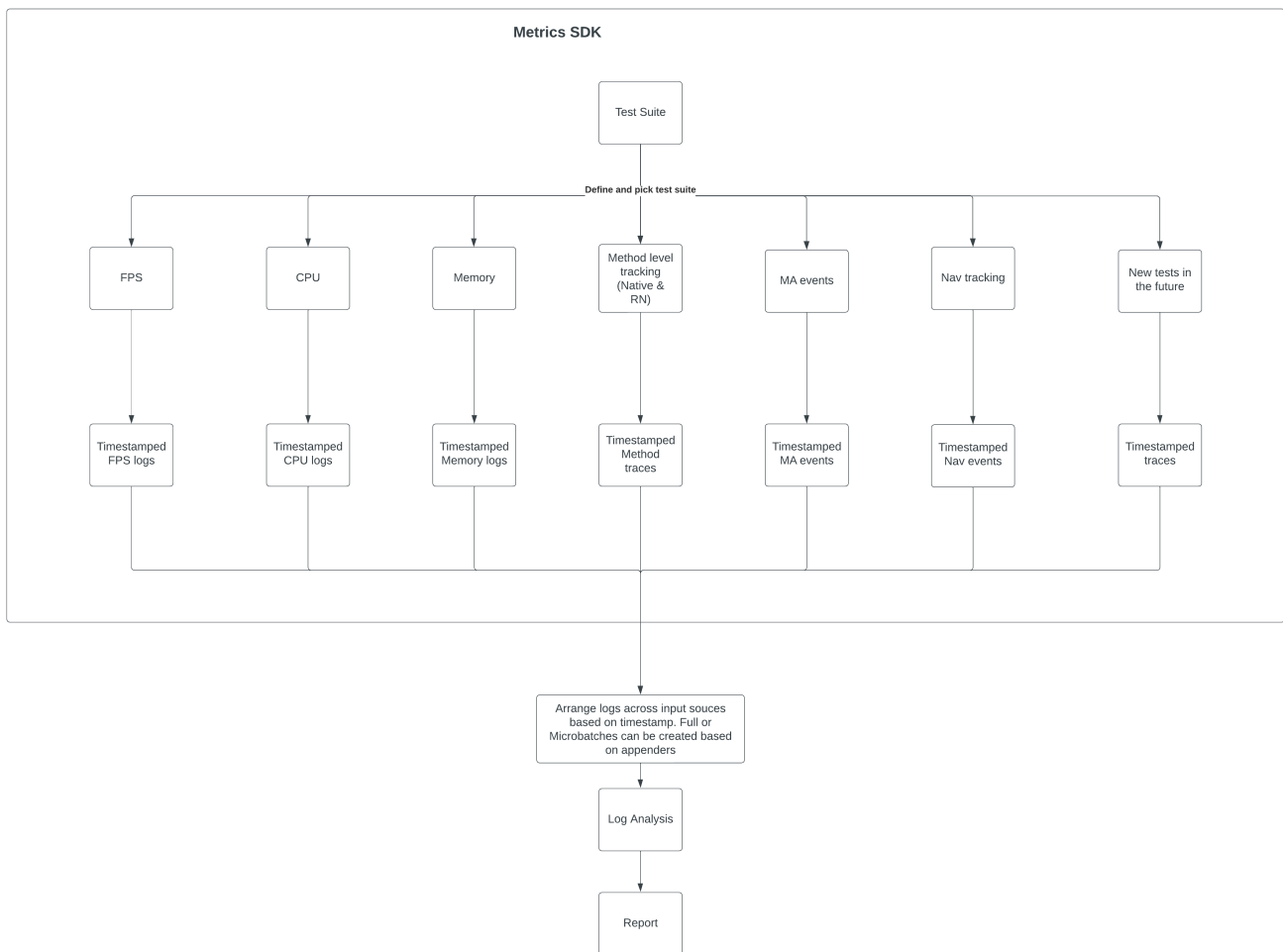
AOP based

1. Log augmentation
 - a. Capture user path
 - b. User interactions
 - c. Method call stack can be automatically added too
 - d. Hard to debug RN crashes can be detected
2. Method call time
 - a. How much time did a method take
3. Assign methods or classes to features
 - a. Tag crashes on a feature or Pod level
 - b. 100 pc accuracy in attribution can't be guaranteed, but very close attribution

Currently UDP dashboard can help in finding what screen was opened etc, but this approach goes way beyond that and adds a lot more details.

NFR aspect?

- Events can be collected batch etc in one payload and sent all at once



//TODO Explanation for above

FE Navigation

FE navigation can be triggered by

- UI Automation tests
- Bot based Navigation
- In Prod setup, these will be actual user clicks and actions

Automated Test Suite

- Helps in detecting issues in critical or important paths
- Core metrics are of utmost importance here

Macrobenchmark or other tooling

Bot based Navigation

- Since number of pages in app can be incredibly large, any one screen can lead to other pages, automated test suite for all can't be written for all
- Exploratory navigation fills in here
- Helps in detecting hard to find issues which might be impacting stuff which no one raises an alarm about.
- Write a bot / custom code which reads layouts as a graph

For navigation a *DFS based approach with max depth of 2-3 can be followed* starting with HP or other screen.

Unbounded DFS in general may not be suitable as we will end up going too deep in screen stacks and may not be able to cover screen navigations which are easily accessible.

Methodology

It will work as follows

1. Test runner clicks on clickable item on screen
2. Tracks screen name / url, was it visited before? If yes, ignore pop back
3. Tracks screens in backstack
4. Monitors changes in core metrics
5. Any breach observed raise an alarm

How do events get sent from FE?

- FE SDK backed by KMM & SQLDelight
- Same SDK to be used by MA & this one with both implementing the abstract classes

Launch phases

Phase 1

In the first phase we

Focus on Identification of issues

- Launch on Headspin Farm
- Integrate with CI & CD
- Daily runs (frequency can be defined)

Phase 2

Streamlining the process & build a few capabilities

- Enhance and augment capabilities based on feedback and issues in Phase 1
- Incorporate more and more event types into the system

The duration of this phase may turn out to be small.

Phase 3

We move towards identification from detection.

- Aim is to identify the causes automatically
- Based on RCAs observed and learnings from previous phases

Phase 4

- Launch as a part of Production apps
- Send metrics data to backend servers

Other sources of Data that can be leveraged

- Firebase Crashlytics through BigQuery

Backend

From the backend NFR perspective, it is important to control the amount of data coming from the device.

Possibility through Morpheus

This can be done through Morpheus, enabling a test bucket. Let the users in the test bucket provide data, rest ignore.

Open question: How can the audience in the test bucket be changed? Decrease of % not possible?

Top level switch flag then Morpheus bucket.

- MA events sent from app for the page load and other metrics identified in the doc:
- Events get fed to Grafana and graphs are generated for top level metrics across all devices and manufacturers etc
- For detailed analysis stream based jobs can be setup feeding off NRT events
 - These jobs can segment and notice specific patterns if need be

Option 1:

MDP

Pros:

- Existing pipeline can be reused, lesser effort
- Realtime anomaly detection can happen through NRT based pipelines

Cons

- MDP to scale further to accommodate this data

Option 2:

New Perf service

Pros:

- More granular control

Cons:

- Requires larger effort to build

Solve

Evolution of SDK

SDK will be developed in Kotlin & KMP. This will allow us to reuse the code as and when we bake it into Production app.

Types of Appenders

- File
- Console / Logcat
- Persistence
- Network

File

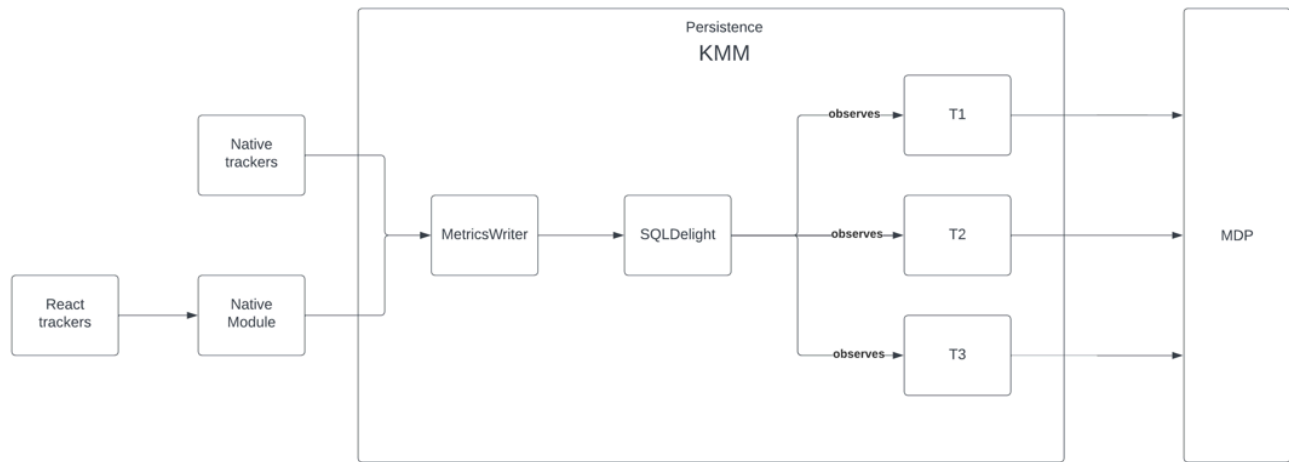
A simple file is used to dump the logs. Optimisations around storing in memory and dumping at intervals, log rotation etc can be done.

Console / Logcat

Utilise platform specific Console & Logcats. Write data there. Read logs from there based on tags. Risk of losing log if max size is breached.

Persistence

Utilise a custom persistence mechanism

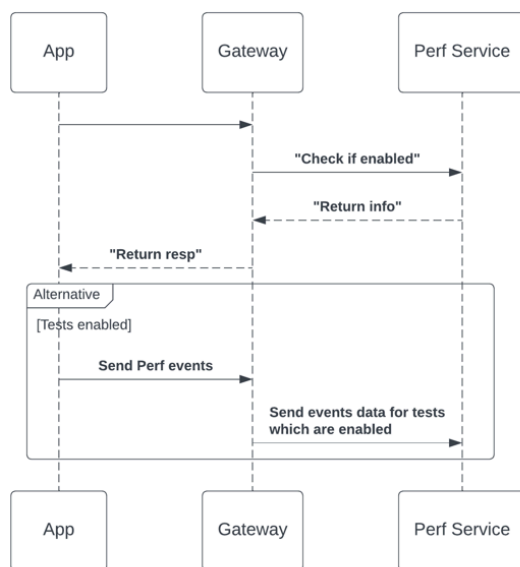


MDP

On Premises vs Realtime perf use cases

- Common SDK
- Lightweight CPU metrics can be collected on Prod vs On Premises

Sequence diagram to be updated as backend services are defined



Automated tests pipeline

- Memory profile
- CPU profiles
- RN profiles (JS thread and other parameters)

- Method call times.
 - Backed by AOP
 - SF Tech offsite link:
- FPS
- Rerenders numbers
- Bridge calls
- Navigation time
- Other validations like
 - Clickable item not responding [Could have helped situations like Search screen freezing]

The parameters to track these on screen

Above parameters to be captured as a part of build generation pipeline and daily automated test suite execution.

When to capture?

- New Build generation
 - To capture if any delta has been added
 - Capture if any improvement or degradation?
- Periodic test execution
 - On Prod apps, to check if any config issue or any service related issues are causing degradation
 - Daily or hourly etc
- Merges in stable develop & release branches

When to raise alerts?

- Diff to be computed from previous data and if it crosses a threshold (x %) alerts to be triggered.
- Core metric shaken: Red alert

Automated tests to run on a few **fixed selected** test devices covering variation

- Android/iOS version
- Manufacturer

This to ensure consistency in test results for a given device set.

Pre test execution, points to validate

- Battery percentage or connected to charger
 - Low battery will cause battery saver settings, CPU will favor energy saver cores leading to lower performance
- Device temperature

- In case of high temperature test to be delayed until device falls back into normal operating temperature
- Thermal throttling status
 - Newer Android versions have thermal throttling level, this api can be used to decide if device can start the tests or not
- Network type and speed
 - 4G or Wifi can impact performance
 - Signal strength
 - Even with Wifi speed matters, router may give full signal but speed may be bad

Test sequence diagram

Regressions in CI/CD

Detecting regressions is important and controlling false positives is important too.

Threshold based approach

Send an alert if the threshold has been breached.

Cons

- Multiple small changes within the thresholds may go undetected. Perf may degrade substantially over a period of time without getting detected.
- Random blip in a test because of external factors may cause false alarms

Avoiding False positives

Setting the right threshold is extremely important

Avoiding False positives:

Use Step Fit algorithm to avoid False positives.

<https://medium.com/androiddevelopers/fighting-regressions-with-benchmarks-in-ci-6ea9a14b5c71>

<https://developer.android.com/topic/performance/benchmarking/benchmarking-in-ci>

Section WIP